

Artificial Intelligence

Course No: 0613-401

Lab-02

NumPy, Pandas & Uninformed Search

Ekramul Haque Amin

Lecturer, AI and Data Science, Dhaka International University (DIU)

Objective

- **Use NumPy indexing, axis and statistics**
- **Read a CSV file with Pandas and explore it**
- **Filter, clean, group and create new columns**
- **Generate synthetic data and prepare it for AI**
- **Implement BFS, DFS and UCS in Python**

NumPy: Indexing & Axis

```
import numpy as np
a = np.arange(1, 13).reshape(3, 4)
# [[ 1  2  3  4]
#  [ 5  6  7  8]
#  [ 9 10 11 12]]

print(a[1, 2]) # 7 (row 1, column 2)
print(a[:, 1]) # [ 2  6 10] (whole column 1)
print(a.sum(axis=0)) # [15 18 21 24] column sums
print(a.sum(axis=1)) # [10 26 42] row sums
print(a[a > 6]) # [ 7  8  9 10 11 12]
```

NumPy: Quick Statistics

```
marks = np.array([72, 48, 91, 35])
```

```
print(marks.max()) # 91
```

```
print(marks.argmax()) # 2 (index of the max)
```

```
print(marks.mean()) # 61.5
```

```
print(np.where(marks ≥ 40, "Pass", "Fail"))
```

```
# ['Pass' 'Pass' 'Pass' 'Fail']
```

- **argmax()** picks the predicted class in classification

Our Dataset: students.csv

Name	Dept	Age	Study_Hours	Attendance	Marks	Result
Rahim	CSE	21	5	90	85	Pass
Karim	EEE	22	2	60	48	Fail
Sadia	CSE	20	6	95	91	Pass
Nusrat	BBA	21	missing	80	72	Pass
Tanvir	EEE	23	1	55	35	Fail
Mitu	CSE	22	4	85	missing	Pass
Arif	BBA	20	3	70	66	Pass
Jannat	CSE	21	7	98	94	Pass

Reading a CSV File

- Keep students.csv in the same folder as your notebook

```
import pandas as pd
```

```
df = pd.read_csv("students.csv")  
print(df.shape) # (8, 7) rows, columns  
print(df.columns) # column names  
df.head() # first 5 rows  
df.tail(3) # last 3 rows
```

Exploring the Data

```
df.info()
```

```
# 8 rows; Study_Hours and Marks have only 7 values
```

```
df.describe() # count, mean, std, min, max
```

```
df["Dept"].value_counts()
```

```
# CSE 4, EEE 2, BBA 2
```

```
df["Marks"].mean() # 70.14
```

Selecting Columns & Rows

```
df["Name"] # one column
```

```
df[["Name", "Marks"]] # several columns
```

```
df.loc[0:2, ["Name", "Marks"]] # rows 0 to 2, by label
```

```
df.iloc[0:2, 0:3] # first 2 rows, first 3 columns
```

- loc uses labels: the end is included
- iloc uses positions: the end is excluded

Data Filtering

```
df[df["Marks"] > 80]  
# Rahim, Sadia, Jannat
```

```
df[(df["Attendance"] < 85) & (df["Result"] == "Pass")]  
# Nusrat, Arif
```

```
df[df["Dept"].isin(["EEE", "BBA"])]  
# Karim, Nusrat, Tanvir, Arif
```

```
df[df["Result"] == "Fail"]["Name"] # Karim, Tanvir
```

- **& = AND, | = OR, and wrap each condition in ()**

Handling Missing Values

```
df.isnull().sum()
```

```
# Study_Hours 1, Marks 1
```

```
# Option 1: fill the gaps
```

```
df["Marks"] = df["Marks"].fillna(df["Marks"].mean())
```

```
df["Study_Hours"] = df["Study_Hours"].fillna(df["Study_Hours"].median())
```

```
# Mitu's Marks → 70.1, Nusrat's Study_Hours → 4.0
```

```
# Option 2: drop rows with gaps
```

```
df_clean = df.dropna()
```

New Columns & Sorting

```
def grade(m):  
    if m ≥ 80: return "A+"  
    elif m ≥ 70: return "A"  
    return "B"  
  
df["Grade"] = df["Marks"].apply(grade)  
  
top = df.sort_values("Marks",  
                    ascending=False)  
top[["Name", "Marks", "Grade"]].head(3)
```

Output:

Name	Marks	Grade
Jannat	94	A+
Sadia	91	A+
Rahim	85	A+

Grouping & Summarizing

```
df.groupby("Dept")["Marks"].mean()
```

```
df.groupby("Dept").agg(avg_marks=("Marks", "mean"),  
                        students=("Name", "count"))
```

Dept	avg_marks	students
BBA	69.0	2
CSE	85.0	4
EEE	41.5	2

Data Synthesis: Generate a Dataset

- Real data is small? Generate realistic data to practise and test models

```
np.random.seed(42) # same numbers every run
n = 100
hours = np.random.randint(0, 9, n)
att = np.random.randint(50, 101, n)
noise = np.random.normal(0, 5, n)
marks = np.clip(8*hours + 0.4*att + noise, 0, 100).round()

syn = pd.DataFrame({"Study_Hours": hours, "Attendance": att, "Marks": marks})
syn["Result"] = np.where(syn["Marks"] ≥ 40, "Pass", "Fail")
syn.to_csv("synthetic_students.csv", index=False)
print(syn.shape) # (100, 4) → 82 Pass, 18 Fail
```

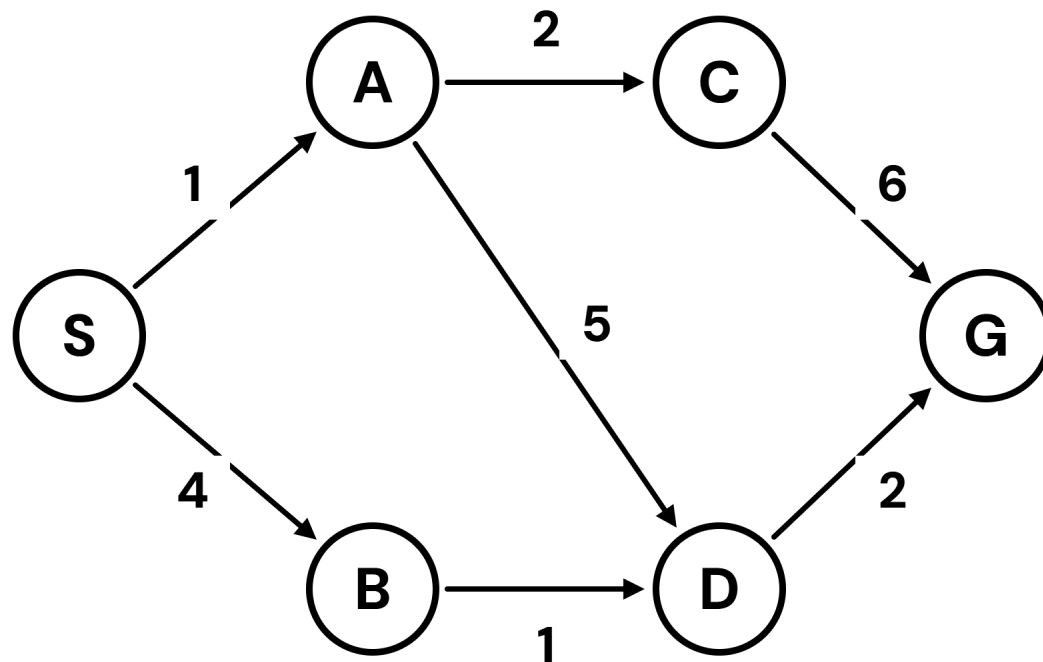
Preparing Data for AI

- **X = features (inputs), y = labels (answers to learn)**

```
X = df[["Study_Hours", "Attendance"]].to_numpy()
y = (df["Result"] == "Pass").astype(int).to_numpy()
print(X.shape) # (8, 2)
print(y) # [1 0 1 1 0 1 1 1]

# Min-Max normalization: scale each column to 0..1
X_norm = (X - X.min(axis=0)) / (X.max(axis=0) - X.min(axis=0))
print(X_norm[:3].round(2))
# [[0.67 0.81] [0.17 0.12] [0.83 0.93]]
```

State Space as a Graph



```
graph = {  
    'S': {'A': 1, 'B': 4},  
    'A': {'C': 2, 'D': 5},  
    'B': {'D': 1},  
    'C': {'G': 6},  
    'D': {'G': 2},  
    'G': {}  
}
```

- Start: S, Goal: G
- Numbers = step costs

Breadth-First Search (BFS)

```
from collections import deque
```

```
def bfs(graph, start, goal):  
    queue = deque([[start]])  
    visited = {start}  
    while queue:  
        path = queue.popleft()  
        node = path[-1]  
        if node == goal:  
            return path  
        for nb in graph[node]:  
            if nb not in visited:  
                visited.add(nb)  
                queue.append(path + [nb])  
    return None
```

- Queue (FIFO): first in, first out
- Explores level by level
- Finds the path with the fewest steps

Depth-First Search (DFS)

```
def dfs(graph, start, goal):  
    stack = [[start]]  
    visited = set()  
    while stack:  
        path = stack.pop()  
        node = path[-1]  
        if node == goal:  
            return path  
        if node not in visited:  
            visited.add(node)  
            for nb in graph[node]:  
                stack.append(path + [nb])  
    return None
```

- **Stack (LIFO): last in, first out**
- **Goes deep along one branch first**
- **The path found depends on neighbour order**

Uniform Cost Search (UCS)

```
import heapq
```

```
def ucs(graph, start, goal):  
    pq = [(0, [start])]  
    visited = set()  
    while pq:  
        cost, path = heapq.heappop(pq)  
        node = path[-1]  
        if node == goal:  
            return cost, path  
        if node in visited:  
            continue  
        visited.add(node)  
        for nb, w in graph[node].items():  
            heapq.heappush(pq, (cost + w, path + [nb]))  
    return None
```

- **Priority queue: lowest cost first**
- **Always expands the cheapest path**
- **Finds the lowest-cost path**

Running All Three

```
print(bfs(graph, 'S', 'G')) # ['S', 'A', 'C', 'G']  
print(dfs(graph, 'S', 'G')) # ['S', 'B', 'D', 'G']  
print(ucs(graph, 'S', 'G')) # (7, ['S', 'B', 'D', 'G'])
```

Algorithm	Path	Cost	Why
BFS	$S \rightarrow A \rightarrow C \rightarrow G$	$1 + 2 + 6 = 9$	Fewest steps, ignores cost
DFS	$S \rightarrow B \rightarrow D \rightarrow G$	$4 + 1 + 2 = 7$	Found by neighbour order, not guaranteed
UCS	$S \rightarrow B \rightarrow D \rightarrow G$	7	Cheapest path, guaranteed

BFS vs DFS vs UCS

Algorithm	Data structure	Complete?	Optimal?
BFS	Queue (FIFO)	Yes	Yes, if all step costs are equal
DFS	Stack (LIFO)	Only in finite graphs	No
UCS	Priority queue	Yes, if costs are positive	Yes, lowest total cost

Tasks

Task-01: Load students.csv. Show the CSE students with Marks above 80.

Task-02: Fill the missing values, add a Grade column and save the result as clean_students.csv.

Task-03: Generate 50 synthetic students. Find the average Marks of the Pass and Fail groups.

Task-04: Add node E to the graph ($B \rightarrow E$ cost 1, $E \rightarrow G$ cost 1). Run BFS, DFS and UCS and compare paths and costs.

Task-05: Change bfs() so it prints each node in the order it is visited.

Viva Questions

1. What does `pd.read_csv()` return?
2. What is the difference between `loc` and `iloc`?
3. How do you find and fill missing values?
4. Why do we generate synthetic data?
5. Which data structure do BFS, DFS and UCS use?
6. Why can BFS return a path that is not the cheapest?
7. When does UCS behave exactly like BFS?

Thank You

Questions?

Ekramul Haque Amin

Lecturer, AI and Data Science, Dhaka International University (DIU)